SYSTEM ARCHITECTURE DOCUMENT

# SAMVED

Safe Access Management & Verification for Entry Decisions

---

## SafeWorkers Platform

Real-time Safety Management for Urban Sanitation Workers

React 19 · Node.js + Express · Socket.IO · Leaflet Maps · Hindi TTS

| Version | Date | Status |
|---|---|---|
| 1.0 | 2025 | **Production Demo** |

Manhole Entry Safety · Sensor Monitoring · AI Advisory · Supervisor Command

# Contents


**1 System Overview** **3**

**2 Technology Stack** **3**

**3 Repository Structure** **4**

**4 High-Level System Architecture Diagram** **5**

**5 Authentication & Authorization** **5**
- 5.1 User Accounts . . . 5
- 5.2 Auth Flow . . . 6

**6 Frontend Architecture** **6**
- 6.1 Routing (`App.jsx`) . . . 6
- 6.2 Global State & Hooks . . . 6
- 6.3 Worker Dashboard (`WorkerDashboard.jsx`) . . . 6
- 6.4 Safety Diary (`SafetyDiary.jsx`) . . . 7
- 6.5 Supervisor Dashboard (`SupervisorDashboard.jsx`) . . . 8
- 6.6 Admin Dashboard (`AdminDashboard.jsx`) . . . 8

**7 Backend Architecture** **9**
- 7.1 Entry Point (`server.js`) . . . 9
- 7.2 REST API . . . 10
- 7.3 Socket.IO Event Reference . . . 10
- 7.4 Sensor Engine (`engine.js`) . . . 11
- 7.5 Gas Simulator (`simulator.js`) . . . 11
- 7.6 Gas Thresholds (`thresholds.js`) . . . 11
- 7.7 Advisory Engine (`evaluateRisk.js`) . . . 12
- 7.8 Safety Profile Controller (`safetyProfile.js`) . . . 12
- 7.9 Plan Evaluator (`evaluatePlan.js`) . . . 12

**8 AI & Innovation Features** **13**
- 8.1 Safety Co-Pilot ( -) . . . 13
- 8.2 AI Ward Analysis (AI ) . . . 13
- 8.3 Safe Route & Team Planning . . . 13

**9 Data Flows** **13**
- 9.1 Worker Entry Flow . . . 13
- 9.2 Supervisor Flow . . . 14
- 9.3 Admin Flow . . . 14

**10 Configuration & Constants** **15**

**11 Seed Data** **15**
- 11.1 Workers (`backend/data/seed.js`) . . . 15

11.2 Jobs (`backend/data/seed.js`) — 4 records . . . . . 16
11.3 Safety Diary (`backend/data/safetyDiary.js`) . . . . . 16

**12 Key Architectural Observations** **16**

**Appendix: Badge Definitions** **17**

## 1. System Overview

**SAMVED** (SafeWorkers) is a full-stack web application built to manage the safety of urban sanitation workers performing hazardous confined-space entry operations — specifically, descending into and working inside manholes. The platform connects three roles in real time:

- **Workers** — follow a guided phase-by-phase entry protocol on a mobile-first dashboard, receive Hindi safety advisories, and can trigger emergency SOS.
- **Supervisors** — monitor all workers on a live map, review PPE photos, manage job dispatch via drag-and-drop, and issue evacuation commands.
- **Admins** — review compliance dashboards, audit logs, incident records, a gamification leaderboard, and AI-powered ward-level analysis.

> The system is a **hackathon / demo deployment** — authentication is client-side only, all data is in-memory, and there is no persistent database. All values reset on server restart.

## 2. Technology Stack

**Table 1:** Full Technology Stack

| Layer | Technology | Version | Notes |
|---|---|---|---|
| Frontend framework | React | 19.x | Functional components, hooks |
| Build tool | Vite | 7.x | Dev server on port 5173/5174 |
| Routing | react-router-dom | v7 | `BrowserRouter`, `PrivateRoute` |
| Real-time (client) | socket.io-client | 4.7.5 | Module-level singleton |
| Maps | react-leaflet / Leaflet | v5 / 1.9 | CARTO Voyager tiles |
| Charts | Recharts | v3.7 | BarChart, LineChart, custom heatmap |
| Icons | lucide-react | 0.575 | Tree-shaken SVG icons |
| Backend runtime | Node.js (CommonJS) | 18+ | `require()` modules |
| Backend framework | Express | 4.18 | REST + middleware |
| Real-time (server) | socket.io | 4.7.5 | Broadcast to all clients |
| CORS | cors | 2.8 | Whitelist localhost:5173/5174 |
| Dev runner | nodemon | v3 | Auto-restart on change |
| Database | *None* | — | In-memory seed arrays |
| Authentication | *None (client)* | — | Hardcoded credentials, sessionStorage |

## 3. Repository Structure

```
SafeWorkers/
        backend/
             server.js              # Express entry, Socket.IO hub, port 3001
             routes/
                  workers.js         # GET /api/workers, /gas-check, /:id
                  jobs.js            # GET /api/jobs, /:id
                  alerts.js          # GET /api/alerts  +  prependAlert() export
             controllers/
                  safetyProfile.js   # buildSafetyProfile()  Safety Diary scoring
             advisory/
                  evaluateRisk.js    # evaluateRisk() + buildWeatherAdvisory()
             sensors/
                  engine.js          # 1-second tick loop  sensor_update + auto_alert
                  simulator.js       # Random-walk per-worker gas state machine
                  thresholds.js      # OSHA/NIOSH limits + classify()
             utils/
                  evaluatePlan.js    # evaluateTodaysPlan() + haversineKm()
             data/
                 seed.js            # 7 workers + 4 jobs (mutated in-memory)
                 safetyDiary.js     # Historical diary data (3 worker profiles)

        frontend/
            vite.config.js
            index.html
            src/
                main.jsx           # React root render
                App.jsx            # Router + PrivateRoute guards
                context/
                     AuthContext.jsx    # Hardcoded users, sessionStorage
                     useAuth.js         # useContext hook
                     ThemeContext.jsx   # Dark/light, localStorage + data-theme
                hooks/
                     useSocket.js       # Module-level socket.io singleton
                components/            # Landing page components only
                     Header / Hero / Features / Challenges
                     TrustBanner / TechStack / Footer
                     ScrollReveal.jsx   # IntersectionObserver fade-in
                     GasGauge.jsx       # Reusable SVG arc gauge (shared)
                pages/
                    Login.jsx
                    worker/
                         WorkerDashboard.jsx   # 4-phase state machine
                         WorkerDashboard.css
                         SafetyDiary.jsx/css   # Diary sub-tab
                    supervisor/
                         SupervisorDashboard.jsx
                         SupervisorDashboard.css
                    admin/
                        AdminDashboard.jsx
                        AdminDashboard.css
```

## 4. High-Level System Architecture Diagram

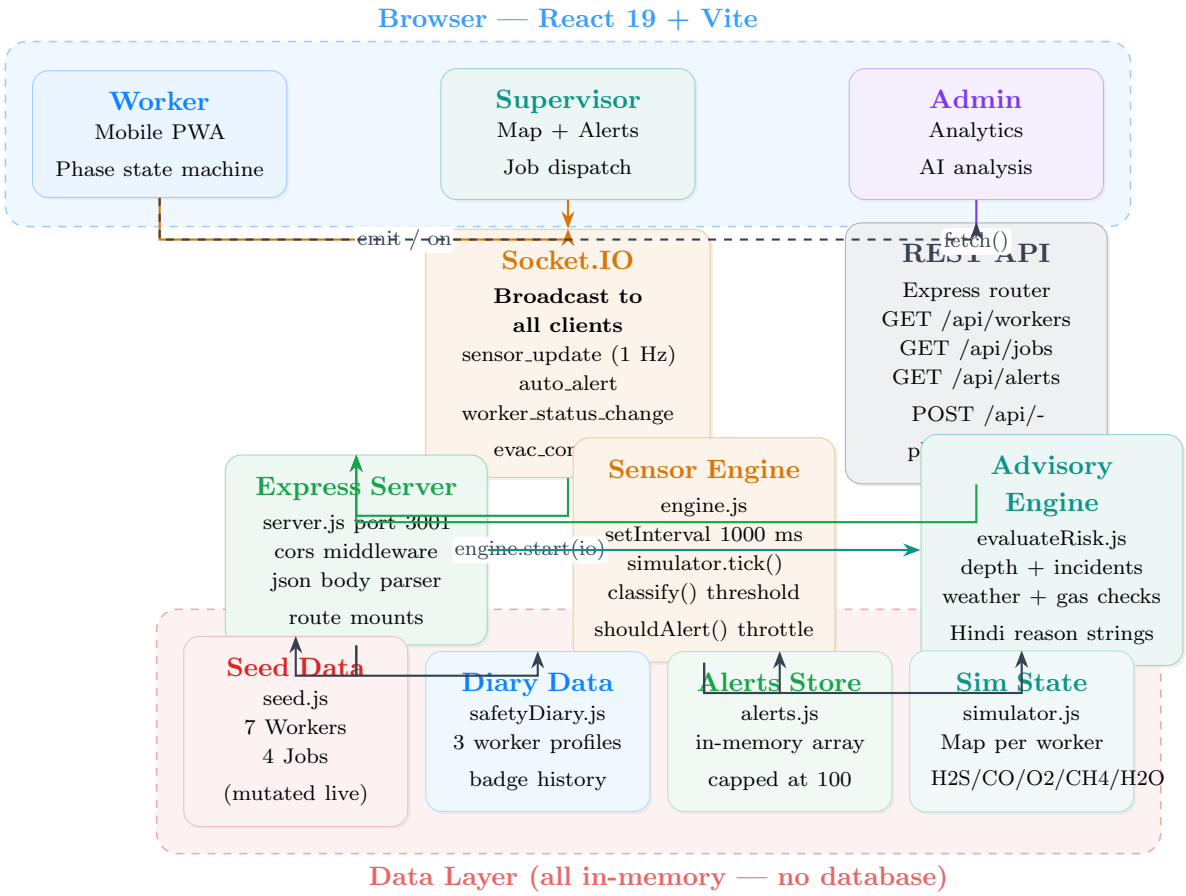


**Figure 1:** SAMVED High-Level System Architecture

## 5. Authentication & Authorization

> **Important:** All authentication is client-side only. No server-side session, JWT, or cookie mechanism exists. Credentials are stored in plaintext in source code.

### 5.1 User Accounts

**Table 2:** Hardcoded Demo Credentials

| Username | Password | Role | Redirect |
|---|---|---|---|
| `ravi.kumar` | `worker123` | worker | `/worker` |
| `priya.sharma` | `super123` | supervisor | `/supervisor` |
| `arjun.das` | `admin2024` | admin | `/admin` |

### 5.2 Auth Flow

1. `login(username, password)` in `AuthContext.jsx` performs an array `.find()` on the hard-coded `USERS` array.
2. On match, the password is stripped and the safe user object is saved to `sessionStorage` under key `samved_user`.
3. `PrivateRoute` in `App.jsx` reads `currentUser` and checks `currentUser.role` matches the required role. Fails redirect to `/login`.
4. `logout()` clears `sessionStorage` and sets `currentUser = null`.

## 6. Frontend Architecture

---

### 6.1 Routing (`App.jsx`)

**Table 3:** Application Routes

| Path | Component | Guard |
|---|---|---|
| `/` | Landing page (full layout) | Public |
| `/login` | `Login.jsx` | Public |
| `/worker` | `WorkerDashboard.jsx` | `role = "worker"` |
| `/supervisor` | `SupervisorDashboard.jsx` | `role = "supervisor"` |
| `/admin` | `AdminDashboard.jsx` | `role = "admin"` |
| `*` | Redirect to `/` | — |

### 6.2 Global State & Hooks

`AuthContext.jsx` Provides `currentUser`, `login()`, `logout()` to the whole tree. Persisted via `sessionStorage`.

`ThemeContext.jsx` Provides `theme` (`"dark"` / `"light"`) and a toggle function. Persisted via `localStorage`. Sets `data-theme` attribute on `document.documentElement`.

`useSocket.js` Module-level singleton socket: `io("http://localhost:3001")` created once on import and shared across all components that call `useSocket()`.

### 6.3 Worker Dashboard (`WorkerDashboard.jsx`)

The most complex page in the application. Implements a 4-phase state machine:

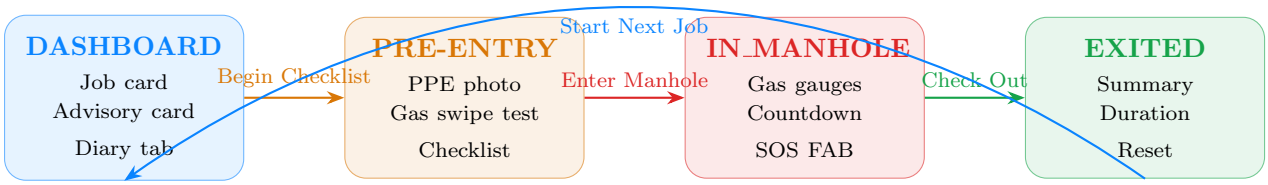


**Figure 2:** Worker Dashboard Phase State Machine

### Key State Variables

**Table 4:** Worker Dashboard State

| State variable | Type | Purpose |
|---|---|---|
| `phase` | enum string | Current phase (DASHBOARD / PRE_ENTRY / IN_MANHOLE / EXITED) |
| `ppeConfirmed` | boolean | Camera photo approved |
| `gasConfirmed` | boolean | Pre-entry gas swipe confirmed safe |
| `gasReadings` | object | Live H2S/CO/O2/CH4/WATER from socket |
| `countdown` | number | Entry timer in seconds (1800 = 30 min) |
| `advisory` | object | Safety Co-Pilot payload |
| `advisoryVisible` | boolean | Show/hide advisory card |
| `onDuty` | boolean | Header duty-toggle state |
| `sosActive` | boolean | SOS modal visible |
| `evacModal` | boolean | Evacuation modal visible |
| `hazardModal` | boolean | Hazard report modal visible |
| `activeTab` | string | `"job"` or `"diary"` |
| `preCheck` | object | Gas-check state: idle / checking / ok / blocked |

### Inline Sub-components

`SwipeConfirm` A drag-to-confirm slider (touch + mouse). Uses `useRef` for the track, calculates drag offset as a percentage, fires `onConfirm` at 82% of track width. Triggers `navigator.vibrate()` on success.

`SosFab` Fixed red FAB requiring a 1.5-second long-press to activate. Uses `requestAnimationFrame` to animate an SVG progress ring. Plays a 660 Hz Web Audio tone and vibrates on completion.

`GasGauge` Reusable SVG arc gauge from `components/GasGauge.jsx`. Renders a donut arc with a colour-coded status (safe / warning / danger).

## 6.4 Safety Diary (`SafetyDiary.jsx`)

Rendered inside the `"diary"` tab of WorkerDashboard. Displays worker gamification:

- **Score arc** — SVG semicircle showing a 0–100 safety score calculated server-side
- **Stats grid** — Total jobs, high-risk completions, hazard reports, checklists (2×2)
- **Rotating message** — One motivational Hindi phrase rotating every 5 minutes via `Math.floor(Date.now() / 300000) % messages.length`
- **Badges** — 7 badge types (earned vs locked) in a vertical list layout
- **Timeline** — Recent jobs with risk pills and PPE/gas pass indicators

**Data flow:** Starts with a `LOCAL_DEMO` seed, then emits `get_safety_profile` over socket and replaces data on `safety_profile` response.

### 6.5 Supervisor Dashboard (`SupervisorDashboard.jsx`)

**Table 5:** Supervisor Dashboard Tabs

| Tab | Contents |
|---|---|
| Alerts | Real-time alert cards (SOS / gas / hazard). Hovering a card flies the map to that worker. |
| PPE | Approval queue for worker PPE photos — Approve or Deny buttons. |
| Roster | Worker list with fatigue bar, battery percentage, signal strength. Click row → map flyTo. |
| Jobs | Drag-and-drop job dispatch. Drag a worker onto a job or a job onto a worker row. |

**Map:** react-leaflet centred on Mumbai (`19.076, 72.8777`), CARTO Voyager CDN tiles. Each worker is a custom `L.divIcon` with live status colour. Popups show live gas pills updated from the `sensor_update` socket event. An **Evacuate** button inside the popup emits `evac_command`.

**SOS modal:** Full-screen overlay driven by `useSosAlarm` hook — alternates 880 Hz/660 Hz tones via Web Audio API. Actions: *Dispatch Emergency* or *Acknowledge & Silence*.

**Fatigue guard:** When drag-assigning a worker whose `entries >= maxEntries`, a warning modal fires with *Cancel* or *Override & Assign*.

### 6.6 Admin Dashboard (`AdminDashboard.jsx`)

Eight tabs, all data is static in-component constants. Admin has no Socket.IO connection.

**Table 6:** Admin Dashboard Tabs

| Tab | Contents |
|---|---|
| `audit` | Searchable, date-filterable audit log. CSV export via `Blob` URL. |
| `health` | Worker safety scores, entry counts, medical checkup status, incident-free streaks. |
| `namaste` | NaMaSTE-SSW compliance: Aadhaar, PM-JAY, training validity, KYC, status filter. |
| `incidents` | Incident review with gas readings at time of incident and override flags. |
| `leaderboard` | Sanipreneur leaderboard with safety ranking and subsidy eligibility. |
| `ai` | AI ward analysis — typewriter animation + Hindi TTS (see Section 8). |
| `plan` | Safe Route & Team Planning — pre-deployment safety validation table. |
| `charts` | Recharts BarChart (entries + incidents by zone), LineChart (weekly trend), heatmap. |

## 7. Backend Architecture

### 7.1 Entry Point (`server.js`)

- Creates an `Express` app, an `http.Server`, and a `socket.io Server` on the same HTTP server.
- Attaches `cors()` and `express.json()` as global middleware.
- Mounts route files and registers inline REST handlers (`/api/safety-profile/:workerId`, `/api/advisory/weather-alert`, `/api/plan/evaluate`).
- Calls `engine.start(io)` to begin the 1-second sensor tick loop.
- Registers all Socket.IO event handlers inline.
- Listens on port **3001**.

### 7.2 REST API

**Table 7:** REST API Endpoints

| Method | Path | Description |
|---|---|---|
| GET | `/api/workers` | Full workers array |
| GET | `/api/workers/gas-check` | One-shot pre-entry sensor snapshot |
| GET | `/api/workers/:id` | Single worker by ID |
| GET | `/api/jobs` | Full jobs array |
| GET | `/api/jobs/:id` | Single job by ID |
| GET | `/api/alerts` | In-memory alert history (cap 100) |
| GET | `/api/safety-profile/:workerId` | Safety Diary profile object |
| POST | `/api/advisory/weather-alert` | Broadcast a weather advisory |
| POST | `/api/plan/evaluate` | Validate today's job assignments |

### 7.3 Socket.IO Event Reference

**Table 8:** Client → Server Events

| Event | Payload | Server Action |
|---|---|---|
| `worker_enter_manhole` | {`workerId`} | Set status → IN_MANHOLE, broadcast `worker_status_change` |
| `worker_exit_manhole` | {`workerId`} | Set status → IDLE, reset simulator, broadcast |
| `sos_manual` | {`workerId, workerName`} | Set status → SOS, broadcast `auto_alert` + `worker_status_change` |
| `evac_command` | {`workerId`} | Broadcast `evac_command` to all |
| `hazard_report` | {`workerId, hazard`} | Broadcast `auto_alert` (HAZARD, warning) |
| `request_advisory` | {`workerId, jobParams`} | Call `evaluateRisk()`, emit `live_safety_advisory` to sender |
| `get_safety_profile` | {`workerId`} | Call `buildSafetyProfile()`, emit `safety_profile` to sender |

**Table 9:** Server → Client Events

| Event | Source | Description |
|---|---|---|
| `sensor_update` | `engine.js` (1 Hz) | {`workerId, readings, statuses, timestamp`} |
| `auto_alert` | `engine.js` / server | Gas threshold crossed, SOS, or hazard report |
| `worker_status_change` | `server.js` | Worker status updated (broadcast to all) |
| `live_safety_advisory` | `server.js` | Safety Co-Pilot advisory object |
| `safety_profile` | `server.js` | Diary profile object (targeted to requesting socket) |
| `evac_command` | `server.js` | Evacuation order (broadcast; worker filters by ID) |

> All socket events are broadcast via `io.emit()` — there are no rooms. Every connected client receives all events and filters by `workerId` on the receiving end.

### 7.4 Sensor Engine (`engine.js`)

Runs a `setInterval` every 1000 ms. Each tick:

1. Filters the workers array for `status === "IN_MANHOLE"`.
2. Calls `simulator.tick(workerId)` → returns updated {`H2S, CO, O2, CH4, WATER`}.
3. Calls `classify(gas, value)` from `thresholds.js` for each reading → `"safe"` / `"warning"` / `"danger"`.
4. Emits `sensor_update` to all clients.
5. For each gas, calls `shouldAlert()` — fires only if severity changed OR 30-second throttle has expired AND severity is not `"safe"`.
6. On alert: calls `prependAlert()` (in-memory, capped at 100) and emits `auto_alert`.

### 7.5 Gas Simulator (`simulator.js`)

- Maintains a `Map<workerId, readings>` of per-worker gas state.
- `tick(workerId)`: each gas walks by a uniform random ±`stepSize`, clamped to physical bounds.
- `reset(workerId)`: deletes worker state on exit.

**Initial safe values:** $H_2S$ = 0.3 ppm, CO = 10 ppm, $O_2$ = 21.0%, $CH_4$ = 3% LEL, WATER = 2 cm.

### 7.6 Gas Thresholds (`thresholds.js`)

Based on OSHA / NIOSH confined-space entry standards:

**Table 10:** Gas Classification Thresholds

| Gas | Safe | Warning | Danger | Unit |
|---|---|---|---|---|
| $H_2S$ | $< 1$ | $1 \leq x \leq 10$ | $> 10$ | ppm |
| CO | $< 25$ | $25 \leq x \leq 200$ | $> 200$ | ppm |
| $O_2$ | 19.5–23.5 | $16 \leq x < 19.5$ | $< 16$ | % |
| $CH_4$ | $< 10$ | $10 \leq x \leq 25$ | $> 25$ | % LEL |
| WATER | $< 10$ | $10 \leq x \leq 30$ | $> 30$ | cm |

### 7.7 Advisory Engine (`evaluateRisk.js`)

Takes {`depth, recentIncidents, weather, gasReadings`} and returns a structured advisory:

```
{
  priority: "high" | "medium" | "low",
  title:    "<Hindi title>",
  reasons:  ["<Hindi reason string>", ...],
  tips:     ["<Hindi tip string>", ...],
  speak:    "<Hindi TTS string>",
  source:   "backend" | "local"
}
```

Priority is determined by `reasons.length`: $\geq 2 \rightarrow$ high, $= 1 \rightarrow$ medium, $= 0 \rightarrow$ low.

**Checks performed:** depth $> 3$m, recent incidents $> 0$, weather in {RAIN, STORM, FLOOD, HEAT}, $H_2S \geq 10$, CO $\geq 200$, $O_2 < 19.5\%$, $CH_4 \geq 25\%$, WATER $\geq 30$ cm.

### 7.8 Safety Profile Controller (`safetyProfile.js`)

Computes the Safety Diary score from historical diary data:

$$rawScore = (highRiskCompleted \times 3) + (proactiveReports \times 2) + checklistCount$$
$$displayScore = \min(100,\ rawScore \times 2.5)$$

Awards 7 gamification badges based on thresholds. Generates Hindi reinforcement messages based on score level and incident-free streak.

### 7.9 Plan Evaluator (`evaluatePlan.js`)

Validates a set of daily job assignments. For each worker, applies three rules:

1. **HIGH_RISK_OVERLOAD** — more than 2 high-risk jobs in one shift.
2. **ROUTE_FATIGUE** — total Haversine route distance exceeds 3.5 km.
3. **WORKLOAD_OVERRUN** — more than 3 total jobs in one shift.

Also exports `haversineKm()` and `routeKm()` helpers (used for rule 2). This logic is duplicated verbatim in `AdminDashboard.jsx` as a client-side fallback with a 2-second timeout.

# 8. AI & Innovation Features

## 8.1 Safety Co-Pilot ( -)

> A dual-source advisory engine ensures the worker always sees a safety advisory — even with no network connection.

1. **Local computation:** `computeLocalAdvisory()` in `WorkerDashboard.jsx` runs synchronously on mount using the hardcoded job parameters. The advisory card is visible instantly.
2. **Backend computation:** Simultaneously, `socket.emit("request_advisory")` fires. The backend `evaluateRisk()` result arrives via `live_safety_advisory` and updates the card.
3. **Audio:** The worker can tap the speaker icon at any time to hear the advisory read aloud via `window.speechSynthesis` in Hindi (`lang: "hi-IN"`, rate 0.88).
4. **Iconographic hazards:** Each reason bullet receives a context-mapped emoji icon ( depth, rain, storm, flood, heat, gas).

## 8.2 AI Ward Analysis (AI )

1. Admin clicks *AI* for a ward (W1–W8).
2. Pre-authored Hindi narratives stored in the `AI_WARD_TEXT` constant render via a typewriter effect: 20 ms per character using chained `setTimeout` calls stored in `typingRefs`.
3. A Volume icon appears. Click to start Hindi TTS (`lang: "hi-IN"`, rate 0.88). Click again (icon becomes `VolumeX`) to stop immediately via `speechSynthesis.cancel()`.
4. Speech state is tracked in `speakState` React state, enabling icon toggling and stop-on-demand.

## 8.3 Safe Route & Team Planning

- Client-side Haversine distance computation (duplicated from backend utility) for offline capability.
- Three safety rules enforced on every *Safety Check* click.
- Swap suggestion algorithm: `suggestSwapForWorker()` finds the worker with the fewest high-risk jobs as the optimal swap target, returning Hindi message strings.
- Backend validation via `POST /api/plan/evaluate` with a 2-second `AbortSignal` timeout; falls back to client-side on failure.

# 9. Data Flows

## 9.1 Worker Entry Flow

```
1. Worker loads /worker
      sessionStorage restores auth session
      computeLocalAdvisory() runs immediately (no network)
      socket.emit("request_advisory")  server   "live_safety_advisory"
```

```
2. PRE-ENTRY phase
      Camera overlay   photo captured   ppeConfirmed = true
      Swipe slider   GET /api/workers/gas-check
          safe: gasConfirmed = true
          blocked: retry swipe visible
      Both confirmed   "ENTER MANHOLE" button unlocks

3. IN-MANHOLE phase
      socket.emit("worker_enter_manhole", {workerId})
      server: worker.status = "IN_MANHOLE"
      engine starts emitting sensor_update every 1s
      Gas gauges render live H2S/CO/O2/CH4/WATER
      Threshold crossed   engine emits auto_alert  hazardModal opens

4. SOS / Evacuation
      Long-press SOS FAB (1.5s)  socket.emit("sos_manual")
      socket.on("evac_command")  evacModal    dismiss calls handleExit()

5. EXIT phase
      socket.emit("worker_exit_manhole")
      server: simulator.reset(), worker.status = "IDLE"
      EXITED screen   summary    "Start Next Job" resets all state
```

## 9.2 Supervisor Flow

```
1. Supervisor loads /supervisor
      Leaflet map renders with 7 coloured worker pins

2. Real-time updates
      socket.on("sensor_update")  gasCache updated   popup gas pills refresh
      socket.on("auto_alert")     alert prepended; if critical  SOS modal opens
      socket.on("worker_status_change")  map pin colour changes

3. SOS response
      useSosAlarm plays 880/660 Hz alternating tones
      "Acknowledge"   worker.status = DELAYED, alarm silenced

4. Job dispatch
      Drag job card onto roster row (or worker chip onto job card)
      doAssign() moves job; worker.status  TRANSIT
      If worker.entries >= maxEntries  fatigueWarn modal fires

5. Evacuation
      Map popup "Evacuate" button
      socket.emit("evac_command", {workerId})
      All clients receive; target worker's device shows evac modal
```

## 9.3 Admin Flow

```
1. Admin loads /admin (no socket connection)

2. AI Analysis
      Click "AI                            " on a ward
```

```
      Loading spinner 1.5s  typewriter from AI_WARD_TEXT
      Volume icon: toggleSpeak() starts/stops Hindi TTS

3. Plan validation
      Click "Safety Check"
      fetch POST /api/plan/evaluate (AbortSignal 2s timeout)
          success: server evaluateTodaysPlan() result
          timeout: fallback to evaluatePlanLocally()
      Issues modal with swap suggestions  "Confirm Deployment"

4. Audit CSV export
      useMemo filtered log (text search + date range)
      Click "CSV Export"  Blob URL   browser download
```

## 10. Configuration & Constants

> There are no `.env` files. All configuration is hardcoded in source files.

**Table 11:** Hardcoded Configuration Values

| Constant | Value | Location |
|---|---|---|
| Backend port | `3001` | `backend/server.js` |
| CORS allowed origins | `:5173, :5174` | `backend/server.js` |
| Socket server URL | `http://localhost:3001` | `hooks/useSocket.js` |
| API base URL | `http://localhost:3001` | Each `fetch()` call |
| Sensor tick interval | `1000 ms` | `sensors/engine.js` |
| Alert throttle window | `30,000 ms` | `sensors/engine.js` |
| Entry timer | `1800 s (30 min)` | `WorkerDashboard.jsx` |
| SOS hold duration | `1500 ms` | `WorkerDashboard.jsx` |
| Plan evaluation timeout | `2000 ms` | `AdminDashboard.jsx` |
| Map centre | `[19.076, 72.877]` | `SupervisorDashboard.jsx` |
| Map tile provider | CARTO Voyager CDN | `SupervisorDashboard.jsx` |
| TTS language | `hi-IN` | Multiple files |
| TTS rate | `0.88` | Multiple files |

## 11. Seed Data

### 11.1 Workers (`backend/data/seed.js`)

7 in-memory worker records, mutated directly by socket event handlers:

**Table 12:** Worker Seed Fields

| Field | Description |
|---|---|
| `id, name, badge` | Identity and badge number |
| `status` | SOS — IN_MANHOLE — DELAYED — IDLE — TRANSIT |
| `job` | Currently assigned job ID |
| `lat, lng` | GPS coordinates for Leaflet map pins |
| `entries` | Entries completed today (vs `maxEntries`) |
| `battery, signal` | Device battery percent and signal bars |
| `elapsed` | Time in current manhole (seconds) |

### 11.2 Jobs (`backend/data/seed.js`) — 4 records

Each job has: `id`, `address`, `zone`, `risk` (HIGH/MEDIUM/LOW), `priority`, `equipment`, `depth` (metres), `recentIncidents`, `weather`.

### 11.3 Safety Diary (`backend/data/safetyDiary.js`)

Rich historical profiles for workers 1–3 (Ravi Kumar, Suresh Babu, Meena Devi). All other worker IDs receive a `DEFAULT_PROFILE` with zero stats.

## 12. Key Architectural Observations

---

1. **No database.** All state is in-memory. Restarting the backend server resets workers, job assignments, alerts, and gas simulator state.

2. **No backend authentication.** Role-based access is enforced only on the React `PrivateRoute` — any client can call any API endpoint directly.

3. **Broadcast-only sockets.** `io.emit()` sends every event to every connected client. There are no Socket.IO rooms. Clients filter events by `workerId` locally. This could become a performance bottleneck at scale.

4. **Duplicated business logic.** `evaluateRisk()` exists in both `advisory/evaluateRisk.js` (backend) and `WorkerDashboard.jsx` (frontend). `evaluateTodaysPlan()` exists in both `utils/evaluatePlan.js` and `AdminDashboard.jsx`. This is intentional for offline resilience but requires double maintenance.

5. **Frontend seed duplication.** `SupervisorDashboard.jsx` contains its own `WORKERS_SEED` and `JOBS_SEED` mirroring the backend. The supervisor does not fetch worker data from the REST API at runtime.

6. **Single socket instance.** `useSocket.js` creates one module-level socket shared across all pages. Components on different routes all share the same connection.

7. **Admin dashboard is entirely static.** It makes one REST call (plan evaluation) but has no real-time socket connection and displays no live data.

## Appendix: Badge Definitions

**Table 13:** Gamification Badge Unlock Conditions

| Icon | Badge | Unlock Condition |
|---|---|---|
| | First Step | At least 1 job completed |
| | Brave Heart | At least 1 high-risk job completed |
| | Eagle Eye | At least 1 proactive hazard report |
| | Checklist Champ | At least 2 checklists completed |
| | 30-Day Streak | Incident-free for 30 or more days |
| | Safety Star | Safety score $\geq 70$ |
| | Elite Worker | Safety score $\geq 90$ |